PYTHON · МОЙ ПИТОМЕЦ / ЧАСТЬ 1

Мой необычный питомец: создаём игру на Python

Эта методичка посвящена текстовой версии игры. Когда она заработает, я предлагаю подключить окно с кнопками по инструкции «Графическая версия». В папке проекта уже есть готовый графический пример `main.py`; исходный текстовый пример хранится отдельно в `pet_example.py`.

Сегодня я предлагаю тебе стать автором маленькой игры. Ты придумаешь существо, о котором игрок будет заботиться десять игровых дней. У существа появятся имя, привычки и реакции на поступки игрока.

В моём примере живёт облачный лис. У тебя может поселиться кто угодно: сонный вулкан, робот-садовник, кот с Сатурна или существо, которого ещё никто не видел. Я показываю, как устроить программу, а её мир придумываешь ты.

Мы будем работать маленькими шагами: **написали немного → сохранили → запустили → проверили**. Не нужно делать всё за один присест. Если сегодня успеешь только основу, сохрани её: продолжить можно на следующем занятии.

Шаг 1. Придумай героя

Сначала я прошу тебя на несколько минут стать писателем. На бумаге ответь на пять вопросов:

1. Кто твой питомец?
2. Где он живёт?
3. Что он ест?
4. Во что любит играть?
5. Какая у него необычная привычка?

Например: «Облачный лис живёт над городом, ест туман, ловит ветер и чихает маленькими радугами».

Выбери одну деталь, которая будет только у твоего героя. Не обязательно придумывать сложную историю: иногда смешная привычка делает персонажа интереснее длинного описания.

Проверь себя: ты можешь одним-двумя предложениями рассказать мне, кто будет жить в твоей игре.

Шаг 2. Создай папку проекта

Папка — место, где мы храним файлы. Файл — отдельная работа: программа, рисунок или текст. Я предлагаю собрать всё, что относится к игре, в одной папке.

1. Нажми **Win + D**, чтобы увидеть рабочий стол. Клавиша Win обычно отмечена значком Windows. В сочетании клавиш удерживай первую и нажимай вторую.
2. Найди папку со своим именем. Дважды щёлкни по ней левой кнопкой мыши.

3. В открывшемся окне щёлкни правой кнопкой по свободному месту.
4. Выбери **Создать** → **Папку**.
5. Напиши `Мой питомец` и нажми **Enter**.

Вот где теперь будет лежать твоя работа:

```
Рабочий стол
└── Твоё имя
    └── Мой питомец
```

Если папка `Мой питомец` уже существует и там есть твои файлы, не удаляй их. Для новой игры создай папку с другим названием, например `Облачный лис`.

Проверь себя: новая папка находится внутри папки с твоим именем.

Шаг 3. Открой папку в Visual Studio Code

1. Открой **Visual Studio Code** через его значок или меню «Пуск».
2. В верхнем меню выбери **Файл** → **Открыть папку**. В английском интерфейсе это **File** → **Open Folder**.
3. В окне выбора перейди на рабочий стол, затем в папку со своим именем.
4. Выдели папку `Мой питомец` и нажми **Выбор папки** или **Select Folder**.
5. Если VS Code спрашивает, доверяешь ли ты файлам, проверь, что открыта именно созданная тобой папка. Для неё можно подтвердить доверие.

Слева должна появиться панель с названием папки. Если её не видно, нажми **Ctrl + Shift + E**.

Я буду называть три части окна:

- **Проводник слева** — список файлов проекта.
- **Редактор в центре** — место, где мы пишем программу.
- **Терминал снизу** — место, где работающая программа разговаривает с игроком.

Шаг 4. Создай файл и запусти первую строчку

1. Щёлкни правой кнопкой по названию папки в левой панели VS Code.
2. Выбери **Создать файл / New File**.
3. Введи `main.py` и нажми Enter. Используй английские буквы. Окончание `.py` означает, что это файл программы Python.
4. В открывшемся файле напиши:

```
print("Здесь скоро поселится мой необычный питомец!")
```

`print()` выводит сообщение. Текст находится между кавычками. Кавычки и скобки должны быть с двух сторон.

5. Нажми **Ctrl + S**, чтобы сохранить файл.

6. Нажми **Ctrl + Shift + P**. Откроется строка поиска команд VS Code.
7. Введи `Python: Run Python File in Terminal` и выбери найденную команду. Она запускает весь открытый файл в терминале.
8. Посмотри вниз: там должна появиться твоя фраза.

Если команды нет или VS Code предлагает установить Python, позови меня. Мы проверим установленный Python и расширение **Python от Microsoft**. Если нужно выбрать Python, я помогу выполнить **Python: Select Interpreter** и выбрать установленный Python 3. Не нужно наугад скачивать программы.

Для следующих запусков можно использовать кнопку с треугольником справа сверху, если при наведении она называется **Run Python File** или **Run Python File in Terminal**. Эти способы запуска описаны в официальной инструкции VS Code (https://code.visualstudio.com/docs/python/run).

Важно: код пишем в центре, а ответы игре вводим в терминале снизу. Это разные места.

Шаг 5. Познакомь игрока с питомцем

Удали только нашу пробную строку. Вместо неё напиши:

```
print("10 дней с необычным питомцем")
print("Я — облачный лис. Я ем туман и люблю ловить ветер.")
name = input("Как назовём питомца? ")
print("Приятно познакомиться! Меня зовут", name)
```

Я предлагаю сразу заменить описание лиса на описание твоего героя. Текст в кавычках можно менять. Слова `print`, `input` и `name` пока оставь такими, как в примере.

Что здесь происходит:

- `input()` задаёт вопрос и ждёт ответ.
- `name` — имя переменной. Представь коробочку с наклейкой `name`, куда программа кладёт введённое имя.
- `=` означает «запомни значение справа в переменной слева».
- В последней строке `print()` выводит сначала текст, затем значение `name`. Запятая разделяет эти части.

Сохрани и запусти программу. Щёлкни в терминале после вопроса, введи имя и нажми Enter. Пока ты не нажал Enter, программа продолжает ждать.

Проверь себя: в приветствии появилось именно то имя, которое ты ввёл. При следующем запуске попробуй другое.

Шаг 6. Добавь показатели и счётчик дней

Под уже написанным кодом, с новой строки, добавь:

```
hunger = 3
energy = 7
mood = 5
day = 1
total_days = 10
```

Я выбрал английские имена переменных, чтобы их было удобно набирать:

Переменная	Что хранит	Как читать показатель
`hunger`	Голод	0 — сыт, 10 — очень голоден
`energy`	Энергию	0 — устал, 10 — полон сил
`mood`	Настроение	0 — грустит, 10 — очень доволен
`day`	Номер текущего дня	Начинаем с первого
`total_days`	Длительность игры	Играем десять дней

Числа пишутся без кавычек, потому что с ними мы будем выполнять вычисления.

Обрати внимание: **высокий голод — плохо, высокая энергия и настроение — хорошо**.

Пока новые строки ничего не печатают. Это нормально: программа только запоминает числа.

Шаг 7. Сделай повторяющиеся игровые дни

Теперь я предлагаю собрать первую работающую основу. После строки `total_days = 10` добавь весь этот блок. Предыдущие строки оставь на месте.

```
while day <= total_days:
    print()
    print("День", day, "из", total_days)
    print("Голод:", hunger, "/ 10 — меньше значит лучше")
    print("Энергия:", energy, "/ 10 — больше значит лучше")
    print("Настроение:", mood, "/ 10 — больше значит лучше")

    print("1 — Покормить туманом")
    print("2 — Поиграть в ловлю ветра")
    print("3 — Уложить спать на облаке")
    choice = input("Введи 1, 2 или 3 и нажми Enter: ")

    if choice == "1":
        print(name, "съел миску тумана. Вкусно!")
    elif choice == "2":
        print(name, "поймал ветер за хвост!")
    elif choice == "3":
        print(name, "свернулся клубочком на облаке.")

    day = day + 1
```

Разберём его спокойно:

- `while` означает «пока». Пока номер дня не больше длительности игры, повторяем строки внутри цикла.
- `<=` означает «меньше или равно». Поэтому десятый день тоже входит в игру.
- `print()` без текста делает пустую строку для удобства чтения.
- `choice` хранит ответ игрока.
- `==` сравнивает значения. Не путай его с `=`, который записывает значение.
- `if` проверяет первый вариант, `elif` — следующий вариант, если предыдущий не подошёл.
- `day = day + 1` увеличивает номер дня: после первого наступает второй.

Почему в проверке написано `"1"`, а не `1`? `input()` возвращает текст. Мы сравниваем ответ с текстом `"1"`, `"2"` или `"3"`. Превращать его в число здесь не нужно.

Маленький секрет Python: отступы

В начале строк есть пробелы. Они показывают, какие команды относятся друг к другу:

- `while` начинается у левого края;
- обычные команды внутри него — с **четырёх пробелов**;
- команды внутри `if` и `elif` — с **восьми пробелов**;
- последняя строка `day = day + 1` — снова с **четырёх пробелов**.

VS Code часто добавляет отступ сам после Enter. Посмотри, где находится курсор, прежде чем добавлять ещё пробелы. После `while`, `if` и `elif` нужно двоеточие `:`.

Проверь себя: сохрани и запусти игру. Выбирай команды от 1 до 3. Должны пройти дни с первого по десятый. Пока числа не меняются — мы добавим это следующим шагом. Пока вводи только предложенные команды.

Если нужно остановить игру раньше, щёлкни в терминале и нажми **Ctrl + C**. Сообщение `KeyboardInterrupt` после такой остановки ожидаемо. Перед новым запуском заверши предыдущую игру или останови её.

Шаг 8. Пусть действия меняют состояние

Найди внутри цикла участок от `if choice == "1":` до сообщения о сне включительно. **Замени только этот участок** следующим кодом. Строку `day = day + 1` оставь под ним.

```
    if choice == "1":
        hunger = hunger - 3
        print(name, "съел миску тумана. Вкусно!")
    elif choice == "2":
        mood = mood + 2
        energy = energy - 2
        print(name, "поймал ветер за хвост!")
    elif choice == "3":
        energy = energy + 3
        print(name, "свернулся клубочком на облаке.")
```

Например, `hunger = hunger - 3` означает: возьми прежний голод, вычти 3 и запомни новый результат. Было 3 — стало 0.

Наши правила:

Действие	Изменение
Еда	Голод уменьшается на 3
Игра	Настроение растёт на 2, энергия уменьшается на 2
Сон	Энергия растёт на 3

Измени реплики и названия действий под своего героя. Пока я советую оставить числа из примера, чтобы проще проверить работу.

Проверь себя: начни новую игру и выбери еду. В начале второго дня голод должен стать 0. Затем попробуй другие действия. Пока числа могут выходить за границы — следующим шагом мы это исправим.

Шаг 9. Добавь вечер и границы показателей

За день питомец немного проголодается и заскучает. Найди строку `day = day + 1`. **Прямо перед ней**, после всех трёх веток действий, вставь:

```
    hunger = hunger + 1
    mood = mood - 1

    hunger = max(0, min(10, hunger))
    energy = max(0, min(10, energy))
    mood = max(0, min(10, mood))

    print("К вечеру: голод —", hunger, "энергия —", energy, "настроение —", mood)
```

Все эти строки начинаются с **четырёх пробелов**. Они относятся к целому дню и выполняются после любого действия, а не только после сна.

Как работают границы:

- `min(10, hunger)` выбирает меньшее число: если голод оказался 12, получится 10.
- `max(0, ...)` выбирает большее между нулём и полученным результатом: если получилось −2, станет 0.

Сначала выполняется то, что во внутренних скобках, затем то, что во внешних. В итоге показатель остаётся от 0 до 10.

Я выбрал такой порядок: **действие → изменения к вечеру → ограничение чисел → показ результата**. Например, в первый день после еды: `3 - 3 + 1 = 1`. Настроение станет `5 - 1 = 4`.

Проверь себя: начни игру заново и выбери еду. К вечеру должны получиться голод 1, энергия 7, настроение 4. Поиграй несколько дней: в напечатанных показателях не должно быть чисел меньше 0 или больше 10.

В нашей простой версии питомец не погибает и игра не запрещает действия при нулевой энергии. Сначала мы делаем понятную работающую модель. Более строгие правила можно придумать позже.

Шаг 10. Помоги игроку, который ошибся

Игрок может случайно ввести `7`, слово или просто нажать Enter. Я хочу, чтобы игра спокойно попросила повторить ввод и не потратила день.

Найди строку:

```
choice = input("Введи 1, 2 или 3 и нажми Enter: ")
```

Сразу после неё, перед `if choice == "1":`**,** добавь:

```
while choice != "1" and choice != "2" and choice != "3":
    print("Такой команды пока нет. Попробуй ещё раз.")
    choice = input("Введи 1, 2 или 3: ")
```

`!=` означает «не равно». `and` означает «и»: все три проверки должны быть верны одновременно.

Эту строку можно прочитать так: «Пока ответ не равен 1, и не равен 2, и не равен 3 — спрашивай ещё раз».

Получился маленький цикл внутри большого. Большой считает дни, маленький повторяет вопрос. У маленького `while` четыре пробела слева, у двух команд внутри него — восемь.

Проверь себя: введи `кот`, затем `9`, затем нажми Enter без ответа. Игра должна каждый раз повторять вопрос. После ввода `1` должно выполниться кормление, а день увеличиться только один раз.

Шаг 11. Добавь характер

Теперь я предлагаю дать питомцу собственный голос. Найди строку, которая выводит «К вечеру...». **После неё, но перед** `day = day + 1`**,** добавь:

```
if hunger >= 8:
    print(name, "смотрит на твой рюкзак: а он съедобный?")
if energy <= 2:
    print(name, "зевает: даже ветер сегодня слишком быстрый.")
if mood >= 7:
    print(name, "рисует в небе сердечко!")
```

`>=` означает «больше или равно».

Здесь я использую три отдельных `if`. Питомец может одновременно устать и проголодаться, поэтому ему разрешено сказать несколько реплик.

Твоя творческая задача: придумай три собственные реакции. Как разговаривает твой герой: вежливо, загадками, смешными короткими фразами? Можно показывать действия вместо слов: «Робот печатает на лбу надпись: НУЖНА ЗАРЯДКА».

Проверь себя: несколько раз выбери игру. Энергия снизится, и должна появиться реплика об усталости.

Шаг 12. Напиши итог приключения

Перейди в самый конец файла, после строки `day = day + 1`. Начни новые строки **от левого края, без пробелов перед ними**. Если VS Code оставил отступ, убери его клавишей Backspace.

Добавь:

```
print()
print("Наши", total_days, "дней закончились!")
print("Итоговый голод:", hunger)
print("Итоговая энергия:", energy)
print("Итоговое настроение:", mood)

if mood >= 7:
    print("Твоё звание — Легенда весёлых приключений!")
elif energy >= 7:
    print("Твоё звание — Хранитель облачных снов!")
else:
    print("Твоё звание — Исследователь лисьих привычек!")

print("До новой встречи! — говорит", name)
```

Почему начинаем у левого края? Эти команды должны выполниться **после всех дней**, а не каждый вечер.

В цепочке `if / elif / else` выполняется только одна ветка. Сначала проверяется настроение. Если оно не подходит, проверяется энергия. Если не подошло ни одно условие, выполняется `else` — «иначе».

Я использую звания как забавное описание получившегося приключения. Это не оценка того, хорошо ли игрок заботился о питомце по всем показателям.

Придумай свои три звания и короткие прощальные фразы. Например, герой может пригласить игрока на лунный пикник или подарить ему невидимую шляпу.

Проверь себя: итог появляется один раз, после десятого дня. Одиннадцатого дня нет. Если итог печатается каждый день, проверь отступы.

Шаг 13. Сделай игру своей

Основа готова. Я предлагаю сначала сохранить её, а затем выбрать одно-два улучшения. Не нужно делать всё: мне интересна твоя идея, а не количество строчек.

Перед экспериментами сделай запасную копию: нажми Ctrl + S, в Проводнике VS Code щёлкни правой кнопкой по `main.py` и выбери «Копировать», затем правой кнопкой по папке проекта — «Вставить». У копии через «Переименовать» задай имя `main_backup.py`. Продолжай работать во вкладке **main.py**. Резервную копию пока не меняй.

Вариант А. Портрет из символов

Сразу после первого приветствия, **до** `name = input(...)`, добавь:

```
print("(=^.^=)")
print(" (   )")
print("  |_|")
```

Или придумай лицо робота:

```
print("[ o_o ]")
print("[ : : ]")
print(" d   b")
```

Скобки, точки и другие символы внутри кавычек становятся рисунком. Попробуй изменить глаза или выражение лица. Каждая команда `print()` рисует новую строку.

Вариант Б. Особенный пятый день

Внутри большого цикла найди строку `print("День", day, "из", total_days)`. **Перед ней**, после пустого `print()`, вставь:

```
    if day == 5:
        print("Сегодня праздник! Питомец нашёл воздушный торт.")
        mood = min(10, mood + 2)
```

Событие случится перед обычным действием в пятый день. Придумай свой праздник, находку или гостя. Я сразу ограничиваю настроение числом 10, чтобы дневное состояние тоже оставалось в допустимых пределах.

Вариант В. Разговор после еды

Внутри ветки кормления, сразу после сообщения про еду, добавь:

```
        if hunger <= 2:
            print("Спасибо! Сейчас бы полежать и посмотреть на звёзды.")
        else:
            print("Вкусно! Но я бы ещё перекусил.")
```

Здесь новый `if` начинается с восьми пробелов, а его реплики — с двенадцати. Проверяется голод сразу после еды, до вечернего изменения.

Вариант Г. Собственные правила

Измени одно число: например, пусть сон восстанавливает 4 энергии вместо 3. Сначала предположи, что изменится, затем сыграй и сравни. Меняй по одному правилу, чтобы понимать результат.

Можно придумать своё улучшение. Сначала расскажи мне его одним предложением: вместе найдём способ сделать его уже знакомыми командами.

Шаг 14. Разберись с рисунками

Для этой игры картинки **не обязательны**: она работает текстом в терминале. Я советую сначала закончить программу. Файл с рисунком сам по себе не появится внутри Python-игры: показ графики потребует отдельной работы, которой в нашем задании нет.

Если хочется оформить проект, можно нарисовать портрет для отдельного показа:

1. В своей папке `Мой питомец` создай папку `images` тем же способом, что в шаге 2.
2. Открой меню «Пуск», напиши `Paint` и запусти Paint.
3. Нарисуй героя.
4. Выбери **Файл → Сохранить как → Изображение PNG**.
5. Перейди в свою папку на рабочем столе, затем в `Мой питомец`, затем в `images`.
6. Задай имя `pet.png` и сохрани. Если программа сама добавляет `.png`, достаточно имени `pet`.
7. Открой папку `images` и проверь, что рисунок там есть. Двойной щелчок откроет его для просмотра.

Если хочешь найти готовое изображение, можно воспользоваться поиском на Wikimedia Commons (https://commons.wikimedia.org/wiki/Main_Page), например по словам `fox`, `robot` или `cloud`. Открой страницу понравившегося изображения: там указаны автор и условия использования. Если они непонятны, покажи страницу мне. Скачивай изображение в папку `images` и сохраняй ссылку на его страницу и имя автора в файле `idea.txt`, который мы создадим дальше.

Собственный рисунок особенно хорошо подходит придуманному существу: можно показать именно то, что ты представляешь.

Шаг 15. Проверь игру как испытатель

Теперь я прошу тебя ненадолго стать человеком, который впервые открыл игру. Сохраняй файл перед каждым запуском. Каждый запуск начинает новую игру; прогресс предыдущей игры мы пока не сохраняем.

Поставь галочки после проверки:

- □ В приветствии понятно, кто питомец и что делать.
- □ Имя питомца попадает в сообщения.
- □ Еда, игра и сон меняют нужные числа.
- □ Числа в сообщениях остаются между 0 и 10.
- □ Ввод `9`, слова и пустого ответа не завершает программу и не расходует день.
- □ Дни идут от 1 до 10 без пропусков.
- □ Итог появляется один раз после последнего дня.
- □ Есть собственные описание, реплики и звания.

Если ты оставил исходные правила, я предлагаю три коротких испытания:

Что выбирать все десять дней	Что ожидаем в конце
Только игру — `2`	Настроение 10, энергия 0, первое звание
Только сон — `3`	Настроение 0, энергия 10, второе звание
Только еду — `1`	Голод 0, настроение 0, энергия 7, второе звание

Чтобы проверить третье звание, начни новую игру: в первый день выбери игру `2`, а в остальные девять — еду `1`. В конце настроение будет 0, энергия 5, поэтому сработает `else`.

Если ты добавил события или изменил числа, результат может отличаться. Тогда сначала посчитай ожидаемый результат своих правил и проверь его. Отличие от таблицы само по себе не означает ошибку.

Если что-то не получается

Я не жду, что программа сразу заработает без ошибок. Ошибка — подсказка, какое место стоит посмотреть.

Что случилось	Что я предлагаю проверить
Программа ждёт и ничего не делает	Возможно, она дошла до `input()`. Щёлкни в терминале, введи ответ и нажми Enter.
Внизу нет места для ответа	Запускай через **Python: Run Python File in Terminal**. Отвечай во вкладке **Терминал / Terminal**, а не **Вывод / Output**.
После изменений выводится старый текст	Сохрани Ctrl + S. Проверь, что открыта вкладка `main.py`, заверши предыдущую игру и запусти заново.
`SyntaxError`	Проверь кавычки, скобки и двоеточие в указанной строке и строкой выше. Используй обычные кавычки `"`, а не «ёлочки».
`IndentationError` или `TabError`	Проверь отступы. Используй пробелы; команды одного уровня должны начинаться друг под другом.
`NameError`	Сравни имя переменной с тем местом, где ты её создал. `mood` и `Mood` — разные имена. Проверь английскую раскладку.
Всё время первый день	Убедись, что `day = day + 1` находится в конце большого цикла с четырьмя пробелами.
Игра заканчивается каждый день	У итогового блока не должно быть отступа от левого края.
Показатели меняются только после сна	Вечерние изменения должны иметь четыре пробела слева, а не восемь.
Случайно удалил нужный код	Щёлкни в редакторе и нажми Ctrl + Z: это отменяет последнее изменение.

В сообщении об ошибке найди `line` и число рядом — это номер строки. Номера строк видны слева от кода. Прочитай последнюю строку сообщения, посмотри указанное место и проверь, что менял последним.

Если не получилось разобраться, позови меня и покажи сообщение целиком. Не нужно удалять всю программу.

Шаг 16. Сохрани описание и покажи работу

1. В Проводнике VS Code создай новый файл `idea.txt`, как создавал `main.py`.
2. Напиши обычным текстом, без команд Python:

```
Название моей игры:
Автор:
Мой питомец:
Его необычная привычка:
Как запустить игру: открыть main.py в VS Code и запустить Python-файл в терминале.
Как играть: вводить 1, 2 или 3 и нажимать Enter.
Моя любимая находка в проекте:
Что я хочу добавить потом:
```

3. Заполни строки и нажми Ctrl + S. Если использовал чужое изображение, добавь автора, ссылку на страницу и нужные сведения об условиях использования.
4. Переключись обратно на вкладку `main.py` и сохрани её.
5. Через обычный Проводник Windows открой свою папку на рабочем столе. Проверь, что файлы находятся внутри `Мой питомец`.

В готовой папке будут:

```
Мой питомец
├── main.py           — игра
├── idea.txt          — описание
├── main_backup.py    — запасная копия, если ты её сделал
└── images            — если ты добавлял рисунок
    └── pet.png
```

Для обмена проектом копируй **всю папку**: один щелчок по ней, Ctrl + C, переход в указанную мной общую папку или на флешку, Ctrl + V. Используй копирование, чтобы твоя работа осталась у тебя. Место для передачи я покажу на занятии.

Пригласи соседа сыграть на твоём компьютере или передай копию проекта указанным способом. Пусть он попробует разобраться без твоих подсказок. Затем спроси: «Что тебе запомнилось?» и «Где было непонятно?»

В конце я попрошу тебя показать одну особенность, которой ты гордишься: смешную реплику, необычное существо, рисунок или собственное правило. Мне важно, чтобы в игре была видна твоя идея.